

Intelligent and Communicating Systems, ICS

2nd Year Specialty SIQ G02

Lab Report n°01

Title:

Introduction to using and
manipulating a
microcontroller
(Arduino and sensorrs)

Studied by:

First Name: Abderrahmane

Last Name: GRINE

E_mail: ma_grine@esi.dz

A. Theory

1. Getting familiar with microcontrollers

1.1. Microprocessor vs Microcontroller:

A microprocessor is just a CPU. Its memory and input/output components must be connected externally, which makes the overall circuit larger and more expensive. they primarily follow the **Von Neumann architecture**.

A microcontroller, on the other hand, integrates the CPU, memory, and input/output peripherals on a single chip. This makes it compact and cost-effective. Microcontrollers typically use the **Harvard architecture**.

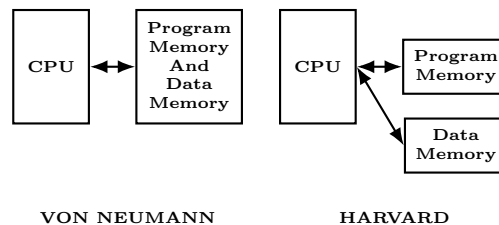


Figure 1: Von Neumann vs. Harvard Architecture

1.2. LED and Pushbutton

The LED is a basic **output** device used for visual indication, while the pushbutton is a digital **input** used to trigger events. They are ideal for testing microcontroller input/output operations and learning how programs interact with hardware.

2. Getting Familiar with Arduino

Arduino is an open-source electronics platform based on easy-to-use hardware and software, widely used for rapid prototyping and embedded system development.

2.1. Arduino MKR WiFi 1010

Each Arduino board has an official **datasheet** that details its specifications, pin configuration, and electrical characteristics, which are essential for proper usage and interfacing. The main features of the MKR WiFi 1010 include:

- **Processor:** SAMD21 Cortex-M0+ (32-bit ARM)
- **Memory:** 256KB Flash, 32KB SRAM
- **Connectivity:** WiFi (NINA-W102), Bluetooth
- **Architecture:** Harvard, enabling separate data and program memory
- **I/O:** Digital/analog pins, PWM, I2C, SPI, UART

2.2. Structure of an Arduino Program

An Arduino sketch includes two main functions:

- `setup()`: runs once at the start, used for initializing the hardware.
- `loop()`: runs repeatedly to execute main program logic.

```

1 void setup() {
2     Serial.begin(9600);
3 }
4 void loop() {
5     Serial.println("Hello , Arduino!");
6     delay(1000);
7 }

```

2.3. Libraries and Functions

The MKR WiFi 1010 mainly uses the `WiFiNINA` library for wireless communication. Other common libraries include `Wire.h`, and `ArduinoHttpClient` for networking, sensor, and IoT integration.

3. Installing and Configuring Arduino Development Tools

3.1. Installing Arduino IDE and connection to TEST Lab

Download and install the Arduino IDE from the official website, making sure to include the necessary USB drivers and board support packages during installation. After installation, connect your Arduino board to the computer using a USB cable. Open the Arduino IDE, go to Tools → Board to select the correct Arduino model, and then choose the corresponding port under Tools → Port. Finally, upload a simple test program (like the “Blink” example) to verify that the setup works correctly.

3.2. Connecting a Pushbutton and LED

Connecting a LED

Connect the LED's anode to a digital pin and its cathode to ground through a resistor. **NB:** In a breadboard, each column of five holes is connected vertically, while the two sides are separated by a gap. The top and bottom rows serve as power (+V) and ground (GND) rails.

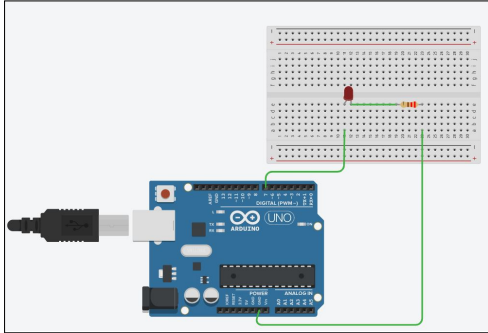


Figure 2: LED blinking circuit on Tinkercad.

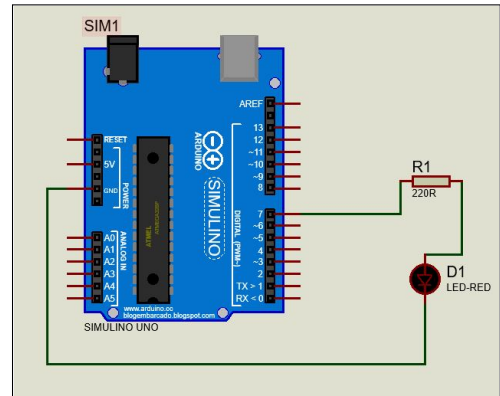


Figure 3: LED schematic diagram on Proteus.

Connecting a Pushbutton

For the pushbutton, connect one side of the pushbutton to digital pin 2 and the other side to ground through a 10 kΩ resistor to maintain a stable LOW state when not pressed. When the button is pressed, the input goes HIGH, turning the LED on pin 13 ON.

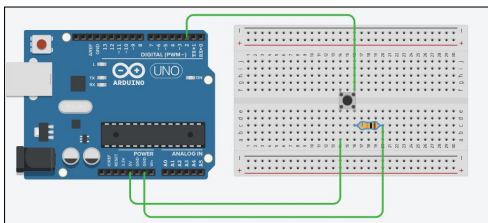


Figure 4: Pushbutton circuit on Tinkercad.

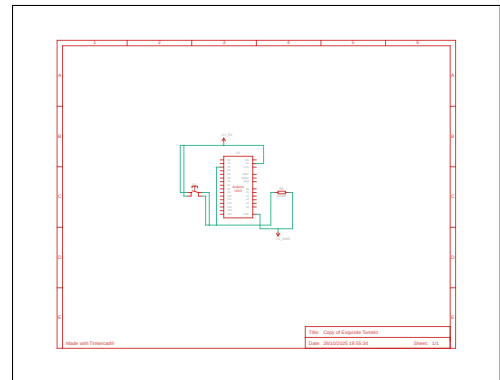


Figure 5: Pushbutton schematic diagram.

3.3. Arduino Protection Mechanisms

To protect Arduino I/O pins, always place a resistor in series with components like LEDs or buttons to limit current. The resistor value depends on voltage and current requirements usually between 200 Ω and 1 kΩ for LEDs. Using Ohm's Law ($R = V / I$), you can calculate the exact value.

4. Introduction to microcontroller simulators

Microcontroller simulators are software tools that allow users to test and debug programs without using physical hardware. Tools like **Tinkercad Circuits** and **Proteus** provide virtual environments for simulating Arduino boards, sensors, and actuators, making them ideal for learning and rapid experimentation.

B. ACTIVITY

1. Arduino Experiments

1.1. Connecting an External LED

Hardware Setup

The circuit uses the **Arduino MKR WiFi 1010** board with one external LED. The cathode ((shorter leg) is connected to the GND through a resistor (between 220 Ω and 330 Ω), while the anode is connected to digital output. This configuration allows the microcontroller to control the LED by sending voltage signals through the output pin.

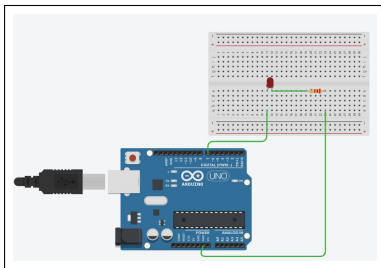


Figure 1.1: LED blinking circuit on Tinkercad.

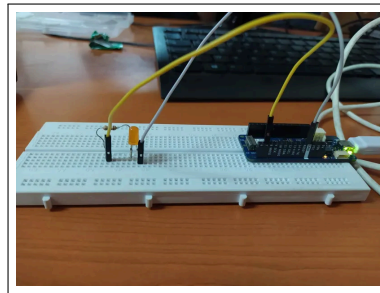


Figure 1.2: Real LED circuit (off state).

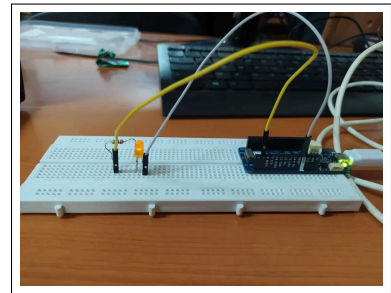


Figure 1.3: Real LED circuit (on state).

Software Design

Before controlling an output device, the pin must be configured as an **OUTPUT**. Once configured, the LED can be switched ON or OFF using digital logic levels. The `delay()` function pauses program execution, controlling the blinking frequency.

```
1 int LED_PIN = 7; // LED connected to digital pin 7
2
3 void setup() {
4   pinMode(LED_PIN, OUTPUT); // Configure pin as output
5 }
6
7 void loop() {
8   digitalWrite(LED_PIN, HIGH); // Turn LED ON
9   delay(2000); // Wait for 2 seconds
10  digitalWrite(LED_PIN, LOW); // Turn LED OFF
11  delay(2000); // Wait for 2 seconds
12 }
```

Listing 1.1: Basic LED control code with comments

1.2. Built-in LED

The same logic applies to the built-in LED connected to `LED_BUILTIN` (pin 6 on MKR WiFi 1010).

```
1 void setup() {  
2   pinMode(LED_BUILTIN, OUTPUT); // Initialize built-in LED pin  
3 }  
4  
5 void loop() {  
6   digitalWrite(LED_BUILTIN, HIGH); // LED ON  
7   delay(2000); // 2-second delay  
8   digitalWrite(LED_BUILTIN, LOW); // LED OFF  
9   delay(2000); // Repeat  
10 }
```

Listing 1.2: Blinking the built-in LED

1.3. Scanning Wireless Wi-Fi Networks

Hardware Setup

The Arduino MKR WiFi 1010 integrates the **NINA-W10** module, which supports **Wi-Fi 802.11 b/g/n** (2.4 GHz) and **qv4.2 (BLE)**. This enables both local wireless communication and internet connectivity, making it ideal for IoT applications and embedded networking tasks.

Software Design

The `WiFiNINA.h` library provides the main interface for wireless operations such as scanning, connecting, and data transmission. It implements standard networking protocols (IEEE 802.11, IP, UDP, HTTP/HTTPS) and allows the Arduino to act as a Wi-Fi client or server.

- `WiFi.scanNetworks()` – scans all available Wi-Fi networks.
- `WiFi.SSID(i)` – retrieves the SSID name of network `i`.
- `WiFi.RSSI(i)` – measures signal strength in dBm.
- `WiFi.encryptionType(i)` – returns encryption type (e.g., WPA2).

In addition, the `MemoryFree.h` library can be used to monitor dynamic memory (SRAM) during program execution. Wi-Fi operations use temporary data buffers for managing network packets and protocol layers, which consume SRAM. Monitoring available memory helps detect potential memory leaks or buffer overflows that may cause connection issues.

Program Example

The following code scans available Wi-Fi networks and prints their SSID, signal strength, and encryption type. It also reports the remaining free SRAM after the scan operation.

```
1 #include <WiFiNINA.h>
2 #include <MemoryFree.h>
3 void setup() {
4     Serial.begin(9600);           // Initialize serial communication at
4         9600 baud
5     while (!Serial);             // Wait for the Serial Monitor to open
5         (for MKR boards)
6     Serial.println("Scanning for Wi-Fi networks..."); // Print message to
6         Serial Monitor
7
8     int n = WiFi.scanNetworks(); // Perform a Wi-Fi scan and store the
8         number of networks found
9
10    // Loop through each detected network and print its details
11    for (int i = 0; i < n; i++) {
12        Serial.print(i + 1);      // Display network number
13        Serial.print(") ");      // Formatting
14        Serial.print(WiFi.SSID(i)); // Print network SSID (name)
15        Serial.print("\tSignal: "); // Label for signal strength
16        Serial.print(WiFi.RSSI(i)); // Print signal strength in dBm
17        Serial.print(" dBm\tEncrypt: "); // Label for encryption type
18        Serial.println(WiFi.encryptionType(i)); // Print encryption type (e.g.,
18            WPA2)
19    }
20
21    // — Memory management section —
22    Serial.print("Free memory: "); // Label for memory information
23    Serial.print(freeMemory());    // Print the amount of free SRAM (in
23        bytes)
24    // Note: The memory check helps verify that Wi-Fi operations
25    // did not consume excessive dynamic memory (important for stability).
26 }
27
28 void loop() {
29     // Empty loop the scan is performed once at startup.
30     // Could be used to repeat the scan periodically (e.g:every few seconds).
31 }
```

Listing 1.3: Wi-Fi network scanning with detailed comments

Implementation

After uploading the program, open the **Serial Monitor**. The Arduino will scan nearby Wi-Fi networks and display their SSIDs, signal strengths, and encryption types. Finally, the available memory in bytes will be printed, showing how much SRAM remains after the scanning process.

2. Conclusion

This activity provided hands-on experience with both hardware interfacing and wireless communication using the Arduino MKR WiFi 1010 board. The first part focused on controlling external and built-in LEDs, reinforcing fundamental digital I/O concepts such as pin configuration, current limitation through resistors, and timing control using the `delay()` function. These experiments demonstrated how the microcontroller generates digital signals to drive output devices and illustrated the importance of proper circuit connections for stable operation.

In the second part, the Wi-Fi scanning experiment introduced the use of the `WiFiNINA.h` library to explore wireless connectivity. The Arduino successfully detected nearby Wi-Fi networks, retrieved their SSIDs, signal strengths (RSSI), and encryption types. By integrating the `MemoryFree.h` library, memory monitoring added to analyze how network operations affect SRAM usage, highlighting the importance of efficient resource management in IoT devices.

Overall, this activity demonstrated the synergy between hardware control and network communication on a single embedded platform.